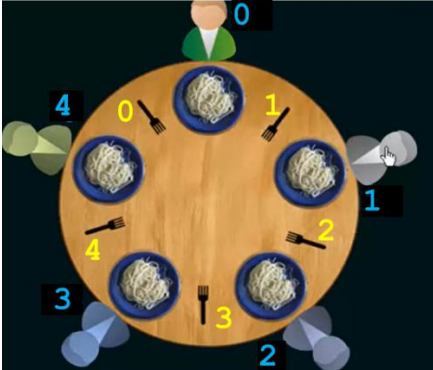


Lab 5: Deadlocks

Total points: 100

Assignment 1 (25 points) – Dining Philosophers 1



Five philosophers are seated around a circular table. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork.

The life of a philosopher consists of alternating periods of eating and thinking. When a philosopher gets sufficiently hungry, she tries to acquire her left and right forks, one at a time, in either order. If successful in acquiring two forks, she eats for a while, then puts down the forks, and continues to think.

The key question is: Can we write a program for each philosopher that does what it is supposed to do and never gets stuck?

Assignment 2 (25 points) – Exercise 8.24 - Dining Philosophers 2

Consider the version of the dining-philosophers problem in which the chopsticks are placed at the center of the table and any two of them can be used by a philosopher. Assume that requests for chopsticks are made one at a time. Describe a simple rule for determining whether a particular request can be satisfied without causing deadlock given the current allocation of chopsticks to philosophers.

Assignment 3 (20 points) – Programming Problem 8.33

Please show disadvantages the following solution.

Account.java ×

```
1 public class Account {
2     private int id;
3     private double balance;
4
5     public Account(int id, double balance) {
6         this.id = id;
7         this.balance = balance;
8     }
9
10    public double getBalance() {
11        return balance;
12    }
13
14    public void setBalance(double balance) {
15        this.balance = balance;
16    }
17
18    public int getId() {
19        return id;
20    }
21 }
```

Bank.java ×

```
3 public class Bank {
4     private ArrayList<Account> accounts = new ArrayList<>();
5
6     public Bank(int accountNum, int balance) {
7         for(int i = 0; i < accountNum; i++) {
8             Account acc = new Account(i, balance);
9             this.accounts.add(acc);
10        }
11    }
12
13    private Account find(int id) {
14        for(int i = 0; i < this.accounts.size(); i++)
15            if(this.accounts.get(i).getId() == id)
16                return this.accounts.get(i);
17        return null;
18    }
19
20    public boolean transaction(int fromId, int toId, double amount) {
21        Account from = this.find(fromId);
22        if(from == null)
23            return false;
24        Account to = this.find(toId);
25        if(to == null)
26            return false;
27        return this.transaction(from, to, amount);
28    }
```

```

29
30 private synchronized boolean transaction(Account from, Account to, double amount) {
31     if (from.getBalance() < amount)
32         return false;
33     from.setBalance(from.getBalance() - amount);
34     to.setBalance(to.getBalance() + amount);
35     return true;
36 }
37 }

```

In Figure 8.7, we illustrate a `transaction()` function that dynamically acquires locks. In the text, we describe how this function presents difficulties for acquiring locks in a way that avoids deadlock. Using the Java implementation of `transaction()` that is provided in the source-code download for this text, modify it using the `System.identityHashCode()` method so that the locks are acquired in order. You should develop an implementation that each `Account` instance has a `ReentrantLock` and these lock objects are ordered using values returned by the `System.identityHashCode()` method.

```

void transaction(Account from, Account to, double amount)
{
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);

    acquire(lock1);
    acquire(lock2);

    withdraw(from, amount);
    deposit(to, amount);

    release(lock2);
    release(lock1);
}

```

Figure 8.7 Deadlock example with lock ordering.

Assignment 4 (30 points) – Programming Project – Banker’s Algorithm

For this project, you will write a program that implements the banker’s algorithm used in deadlock avoidance, discussed in Section 8.6.3. Customers request and release resources from the bank. The banker will grant a request only if it leaves the system in a safe state. A request that leaves the system in an unsafe state will be denied.

```

1 import java.util.ArrayList;
2
3 public class Banker {
4     private int resourceTypeNum;    //hold the number of resource types
5     private int customerNum;        //hold the number of customers
6
7     private int[] available;         //number of resources for each resource type
8     private int[][] maximum;        //the maximum number of requests for each customer
9                                     //Number of rows: the number of customer,
10                                    //number of columns: the number of resource types
11     private int[][] allocation;     //the number of resources of each type currently allocated to each customer
12                                     //Number of rows: the number of customer,
13                                     //number of columns: the number of resource types
14
15     public Banker(int[] avail, int[][] max, int[][] alloc) throws Exception {
16
17         //The system is a in safe state ?
18
19         public ArrayList<Integer> isSafeState() {
20
21             // customer id requests resources; returns true if the banker can allocate resources for this
22             //customer and leaves a safe state
23
24             public ArrayList<Integer> request(int custId, int[] request) {
25
26                 public int[] getAvailable() {
27
28                 public int[][] getMaximum() {
29
30                 public int[][] getAllocation() {

```

Suppose that a snapshot at time T_0 is shown below:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

- Run the program and show whether the system is in a safe state or not.
- Suppose that P_1 requests (1, 0, 2). Can the request be granted?